# ND06AV1C_SDK 用户手册

# 目录


ND06AV1C_SDK 用户手册
目录
- 1. 代码目录介绍
- 2. I2C通讯
  - 2.1. 默认设备地址
  - 2.2. 修改I2C设备地址（需初始化完成之后才能进行）
- 3. ND06AV1C 初始化配置
  - 3.1. 用户实现函数
  - 3.2. 注册用户实现函数到ND06AV1C SDK
  - 3.3. ND06AV1C 模组启动
- 4. ND06AV1C 测量数据获取
  - 4.1. 获取深度和幅度数据
- 5. ND06AV1C 错误码
  - 5.1 测量结果错误
  - 5.3 ND06AV1C错误寄存器
- 6. ND06AV1C 参数配置和获取
- 7. ND06AV1C 标定
  - 7.1. 频率标定
  - 7.2. offset标定
- 8. ND06AV1C 固件升级
- 9. ND06AV1C 低功耗


## 1. 代码目录介绍

代码目录结构如下，代码中包含了设备参数设置、深度获取、角度获取、标定等功能。

```
├── nd06av1c_device
│   ├── inc
│   │   ├── nd06av1c_comm.h
│   │   ├── nd06av1c_data.h
│   │   ├── nd06av1c_def.h
│   │   └── nd06av1c_dev.h
│   └── src
│       ├── nd06av1c_comm.c     -- ND06AV1C通讯接口源文件
│       ├── nd06av1c_data.c     -- ND06AV1C原始数据接口源文件
│       └── nd06av1c_dev.c      -- ND06AV1C设备配置源文件
├── nd06av1c_examples       -- ND06AV1C获取测量信息例程
│   ├── example.c       -- 简单的获取深度例程
│   ├── platform.c      -- 用户需要实现的接口
│   └── platform.h
```

# 2. I2C通讯

## 2.1. 默认设备地址

```
0x5B
```

## 2.2. 修改I2C设备地址（需初始化完成之后才能进行）

ND06AV1C支持修改i2c设备地址，但由于模组没有对修改后的地址进行保存，xshut拉低之后自动恢复默认i2c地址0x5B.
修改ND06AV1C设备的i2c地址后，切记本地同时要更新成修改后的设备地址。
接口如下：

```
/**
 * @brief ND06AV1C Set Slave Address
 *        设置ND06AV1C的设备地址
 * @param pNxDevice: ND06AV1C模组设备信息结构体指针
 * @param addr: 配置ND06AV1C模组的i2c设备地址
 * @return int32_t
*/
int32_t ND06AV1C_SetSlaveAddr(ND06AV1C_Dev_t *pNxDevice, uint8_t addr)
```

# 3. ND06AV1C 初始化配置

参考`/nd06av1c_examples/platform.c`与`/nd06av1c_examples/example.c`

## 3.1. 用户实现函数

ND06AV1C SDK 预留出来5个接口需要用户实现并且注册到SDK中, 分别是:

```
  /**
  * @brief I2C读n个字节
  *
  * @param   i2c_addr    从机设备地址
  * @param   i2c_read_addr   读寄存器地址
  * @param   i2c_read_data   读数据
  * @return  int32_t
  * @retval  0:成功, 其他:失败
  */
  int32_t i2c_read_nbytes(uint8_t i2c_addr, uint16_t i2c_read_addr, uint8_t
*i2c_read_data, uint8_t len);

  /**
  * @brief I2C写n个字节
  *
  * @param   i2c_addr    从机设备地址
  * @param   i2c_write_addr   写寄存器地址
  * @param   i2c_write_data   写数据
  * @return  int32_t
  * @retval  0:成功, 其他:失败
  */
```

```
  int32_t i2c_write_nbytes(uint8_t i2c_addr, uint16_t i2c_write_addr, uint8_t
*i2c_write_data, uint8_t len);

  /**
  * @brief 延时count毫秒
  *
  * @param   count  输入需要延时时长
  * @return  void
  */
  void delay_1ms(uint32_t count);

  /**
  * @brief 延时10*count微秒
  *
  * @param   count  输入需要延时时长
  * @return  void
  */
  void delay_10us(uint32_t count);

  /**
  * @brief 返回当前系统运行时间
  *
  * @return uint64_t
  */
  uint64_t get_now_time_us(void);

  /**
  * @brief 设置nd06av1c xshut引脚电平
  *
  * @param   level  引脚电平, 0为低电平, 1为高电平
  * @return  void
  */
  void set_xshut_pin_level(uint32_t level);
```

## 3.2. 注册用户实现函数到ND06AV1C SDK

```
/* ND06AV1C模组设备结构体 */
ND06AV1C_Dev_t g_nd06av1c_device = {
    .i2c_dev_addr = ND06AV1C_DEFAULT_SLAVE_ADDR,
    .SetXShutPinLevelFunc = set_xshut_pin_level
};

/* 函数指针结构体 */
ND06AV1C_Func_Ptr_t dev_op = {NULL, NULL, NULL, NULL};

/* 初始化函数指针结构体 */
dev_op.Delay10usFunc = delay_10us; /*!< 延时10us函数 */
dev_op.Delay1msFunc = delay_1ms; /*!< 延时1ms函数 */
dev_op.GetNowTimeUsFunc = get_now_time_us; /*!< 获取系统当前运行时间函数 */
dev_op.I2C_ReadNBytesFunc = i2c_read_nbytes; /*!< i2c读函数 */
dev_op.I2C_WriteNBytesFunc = i2c_write_nbytes; /*!< i2c写函数 */

/* 将host端功能函数注册到SDK中 */
ND06AV1C_RegisteredPlatformApi(dev_op);
```

## 3.3. ND06AV1C 模组启动

初始化ND06AV1C之前，请确保已经向SDK注册了操作函数。

- **第一步：调用 *ND06AV1C_WaitDeviceBootUp* 函数**

  *ND06AV1C_WaitDeviceBootUp()*启动函数分为两阶段：

  - 第一阶段：*ND06AV1C_REG_BOOT1~ND06AV1C_REG_BOOT4*寄存器配置启动值；
  - 第二阶段：设置BootLoader启动标志位，选择启动模式；
  - 第三阶段：判断ND06AV1C是否已经进入工作状态，ND06AV1C工作状态寄存器地址为ND06AV1C_REG_STATE,读取到0xA5或者0xA6则说明ND06AV1C工作正常。

```
/**
 * @brief ND06AV1C Wait for Device Boot Up
 *        等待ND06AV1C模组启动
 * @param   pNxDevice   模组设备
 * @param   boot_flag   启动模式
 * @return  ND06AV1C_ERROR_NONE:成功
 *          ND06AV1C_ERROR_BOOT:启动失败--请检测模组是否焊接好，还有i2c地址与读写函数是否错误。
 */
int32_t ND06AV1C_WaitDeviceBootUp(ND06AV1C_Dev_t *pNxDevice,uint32_t boot_flag)
```

- **第二步：调用 *ND06AV1C_InitDevice* 函数**

```
/**
* @brief ND06AV1C InitDevice
*        初始化设备
* @param   pNxDevice   模组设备
* @return  int32_t
*/
int32_t ND06AV1C_InitDevice(ND06AV1C_Dev_t *pNxDevice);
```

  至此，ND06AV1C的启动配置流程就完成了。

# 4. ND06AV1C 测量数据获取

## 4.1. 获取深度和幅度数据

调用 *ND06AV1C_GetDepthAndAmpData* 即可获取**触发一次测量**，并且返回修正之后的16个点的距离数据和幅度数据。函数原型如下：

```
/**
 * @brief ND06AV1C获取深度幅度
 *
 * @param pNxDevice ND06AV1C模组设备
 * @param amp 幅度值
 * @param dep 深度值
 * @return int32_t
 */
int32_t ND06AV1C_GetDepthAndAmpData(ND06AV1C_Dev_t *pNxDevice,uint16_t *amp,uint16_t *dep)
```

dep为传感器到目标点的垂直距离，amp为返回的信号强度。

注：如果结果大于等于65300则表示测量数据错误，详细可以参考ND06AV1C 错误码

# 5. ND06AV1C 错误码

## 5.1 测量结果错误

*ND06AV1C_GetDepthAndAmpData*测量的深度结果错误有以下几种情况，用户需要过滤以下数据：

```
#define ND06AV1C_ERROR_OVEREXP                          65500 /*!< 过曝光 */
#define ND06AV1C_ERROR_UNDEREXP                         65300 /*!< 欠曝 */
#define ND06AV1C_ERROR_OUT_RANGE                        65400 /*!< 不在测距范围 */
```

## 5.3 ND06AV1C错误寄存器

ND06AV1C_REG_ERROR错误标志位寄存器有以下几种，如出现错误则表示模组异常或者标定异常：

```
#define ND06AV1C_ERROR_EXP                              -11 /*!< 自动曝光调节失败 */
#define ND06AV1C_ERROR_CALIB_DATA                       -12 /*!< 没有标定信息 */
#define ND06AV1C_ERROR_DATAERR                          -13 /*!< 原始数据异常 */
```

# 6. ND06AV1C 参数配置和获取

ND06AV1C一般无需配置参数，如用户需要进行参数配置调用*ND06AV1C_SetConfigData*即可设置模组内部参数，调用*ND06AV1C_GetConfigData*可以获取模组内部参数，函数原型如下，部分参数需要进行测距之后才会更新。

```
/**
 * @brief ND06AV1C获取配置数据
 *
 * @param pNxDevice ND06AV1C模组设备
 * @param config 配置项目
 * @param addr_offset 地址偏移量 一般为0
 * @param data 数据
 * @return int32_t
 */
int32_t ND06AV1C_GetConfigData(ND06AV1C_Dev_t *pNxDevice,ND06AV1C_Config_Options
config,uint32_t addr_offset, uint32_t *data)


/**
 * @brief ND06AV1C设置配置数据
 *
 * @param pNxDevice ND06AV1C模组设备
 * @param config 配置项目
 * @param addr_offset 地址偏移量 一般为0
 * @param data 数据
 * @return int32_t
 */
int32_t ND06AV1C_SetConfigData(ND06AV1C_Dev_t *pNxDevice,ND06AV1C_Config_Options
config,uint32_t addr_offset,uint32_t data)
```

部分配置接口已经封装为函数，用户可以自行调用

```
/**
 * @brief 设置offset标定距离
 *
 * @param pNxDevice 模组设备
 * @param calib_dis 标定距离
 * @return
 */
int32_t ND06AV1C_SetCalibDis(ND06AV1C_Dev_t *pNxDevice,uint32_t calib_dis);


/**
 * @brief 设置 or 获取 自动曝光状态
 *
 * @param pNxDevice 模组设备
 * @param status 自动曝光状态 0关闭 1打开 2hdr模式
 * @param set_flag 1 设置 0 获取
 * @; return
 */
int32_t ND06AV1C_SetOrGetAutoExpStatus(ND06AV1C_Dev_t *pNxDevice,uint32_t
*status,uint8_t set_flag);

/**
 * @brief 设置 or 获取 最大积分时间
 *
 * @param pNxDevice 模组设备
 * @param max_exp 最大积分时间
 * @param set_flag 1 设置 0 获取
 * @return
 */
int32_t ND06AV1C_SetOrGetMaxExp(ND06AV1C_Dev_t *pNxDevice,uint32_t
*max_exp,uint8_t set_flag);

/**
 * @brief 设置 or 获取 自动曝光目标幅度值
 *
 * @param pNxDevice 模组设备
 * @param amp_tag 自动曝光目标幅度值
 * @param set_flag 1 设置 0 获取
 * @return
 */
int32_t ND06AV1C_SetOrGetAmpTarget(ND06AV1C_Dev_t *pNxDevice,uint32_t
*amp_tag,uint8_t set_flag);

/**
 * @brief 设置 or 获取 扩展测距模式状态
 *
 * @param pNxDevice 模组设备
 * @param status   状态
 * @param set_flag 1 设置 0 获取
 * @return
 */
```

```
int32_t ND06AV1C_SetOrGetExtendedRangingModeStatus(ND06AV1C_Dev_t
*pNxDevice,uint32_t *status,uint8_t set_flag);

/**
 * @brief 设置滤波器触发时间阈值，两次测距间隔小于此时间则会经过滤波器输出，设置为0关闭此功能
 * @param pNxDevice 模组设备
 * @param time 最大间隔时间
 * @param set_flag 1 设置 0 获取
 * @return
 */
int32_t ND06AV1C_SetOrGetFilterTime(ND06AV1C_Dev_t *pNxDevice,uint32_t
*time,uint8_t set_flag);

/**
 * @brief 设置 or 获取 HDR目标幅度
 *
 * @param pNxDevice 模组设备
 * @param hdr_amp_tag HDR目标幅度值
 * @param set_flag 1 设置 0 获取
 * @return
 */
int32_t ND06AV1C_SetOrGetHDRAmpTag(ND06AV1C_Dev_t *pNxDevice,uint32_t
*hdr_amp_tag,uint8_t set_flag);
```

# 7. ND06AV1C 标定

ND06AV1C模组出厂前已经进行过频率标定和offset标定，但因PCB，电源等因素影响，用户需要重新进行标定，以达到规格书的标准。

## 7.1. 频率标定

模组在使用过程中会受到各种外界因素导致模组内部时钟和预设产生偏差，这时候就需要对频率进行标定。

频率标定需要调用*ND06AV1C_FreqCalib*函数进行标定，函数原型如下：

```
/**
 * @brief   ND06_FreqCalib
 *          ND06频率矫正
 * @param   pNxDevice    模组设备
 * @return  int32_t      0: success
 *                      !0: fail
 */
int32_t ND06AV1C_FreqCalib(ND06AV1C_Dev_t *pNxDevice)
```

**注意：**

- 频率标定依赖*get_now_time_us*准确度，如果此函数不能返回准确的系统时间，则频率标定也会出现偏差，进而影响测距的准确性。
- 如果在ND06AV1C的i2c设备上还挂载了其他设备，在对ND06AV1C进行频率标定时请关闭这些设备。

## 7.2. offset标定

此功能会重新标定传感器的offset，会覆盖出厂标定的数据。

offset标定需要调用*ND06AV1C_OffsetCalib*函数进行标定，函数原型如下：

```
/**
 * @brief ND06AV1C offset标定
 *
 * @param pNxDevice ND06AV1C设备
 * @param err_flag 错误标志位
 * @return int32_t
 */
int32_t ND06AV1C_OffsetCalib(ND06AV1C_Dev_t *pNxDevice,int32_t *err_flag)
```

用户对传感器进行标定流程如下：

1. 调用*ND06AV1C_FreqCalib*函数进行频率标定。
2. 将模组放置于90%反射率平面500mm处，并且平行与此平面，平面大小不小于1000mm*1000mm。
3. 调用*ND06AV1C_OffsetCalib*函数进行offset标定。

**注：err_flag会返回ND06AV1C_REG_ERROR寄存器的值，如果值不为0则表示标定失败**

# 8. ND06AV1C 固件升级

在必要时，FAE工程师会提供模组固件，调用以下函数即可进行固件升级

```
/** ND06AV1C 固件升级 */
int32_t ND06AV1C_FirmwareUpdate(ND06AV1C_Dev_t *pNxDevice,uint8_t
*fw_user,uint8_t *fw_config,uint8_t *fw_calib,uint8_t\ *fw_calib2,uint8_t
*fw_init, uint32_t fw_user_len,uint32_t fw_config_len,uint32_t
fw_calib_len,uint32_t fw_calib2_len,uint32_t\ fw_init_len);
```

# 9. ND06AV1C 低功耗

- **进入硬件待机模式**

  调用*ND06AV1C_HardwareSleep*函数即可进入硬件待机模式；

- **退出硬件待机模式**

  调用ND06AV1C_HardwareWakeup函数即可退出硬件待机模式，此时需要**重新进行ND06AV1C初始化**。